

The basics: Binary numbers

□ Bases we will use

- Binary: Base 2
- Octal: Base 8
- Decimal: Base 10
- Hexadecimal: Base 16

□ Positional number system

- $101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$
- $63_8 = 6 \times 8^1 + 3 \times 8^0$
- $A1_{16} = 10 \times 16^1 + 1 \times 16^0$

□ Addition and subtraction

$$\begin{array}{r} 1011 \\ + 1010 \\ \hline 10101 \end{array}$$

$$\begin{array}{r} 1011 \\ - 0110 \\ \hline 0101 \end{array}$$

Converting decimal numbers to binary

- $53 = 32 + 16 + 4 + 1$
 $= 2^5 + 2^4 + 2^2 + 2^0$
 $= 1*2^5 + 1*2^4 + 0*2^3 + 1*2^2 + 0*2^1 + 1*2^0$
 $= 110101$ in binary
 $= 00110101$ as a full byte in binary

- $211 = 128 + 64 + 16 + 2 + 1$
 $= 2^7 + 2^6 + 2^4 + 2^1 + 2^0$
 $= 1*2^7 + 1*2^6 + 0*2^5 + 1*2^4 + 0*2^3 + 0*2^2 + 1*2^1 + 1*2^0$
 $= 11010011$ in binary

Converting binary numbers to decimal

- What is 10011010 in decimal?

$$\begin{aligned} 10011010 &= 1*2^7 + 0*2^6 + 0*2^5 + 1*2^4 + 1*2^3 + \\ &\quad 0*2^2 + 1*2^1 + 0*2^0 \\ &= 2^7 + 2^4 + 2^3 + 2^1 \\ &= 128 + 16 + 8 + 2 \\ &= 154 \end{aligned}$$

- What is 00101001 in decimal?

$$\begin{aligned} 00101001 &= 0*2^7 + 0*2^6 + 1*2^5 + 0*2^4 + 1*2^3 + \\ &\quad 0*2^2 + 0*2^1 + 1*2^0 \\ &= 2^5 + 2^3 + 2^0 \\ &= 32 + 8 + 1 \\ &= 41 \end{aligned}$$

-
- How do we write negative binary numbers?
 - Historically: 3 approaches
 - Sign-and-magnitude
 - Ones-complement
 - Twos-complement
 - For all 3, the most-significant bit (MSB) is the sign digit
 - 0 $\equiv$ positive
 - 1 $\equiv$ negative
 - twos-complement is the important one
 - Simplifies arithmetic
 - Used almost universally

Binary → hex/decimal/octal conversion

□ Conversion from binary to octal/hex

- Binary: 10011110001
- Octal: 10 | 011 | 110 | 001 = 2361_8
- Hex: 100 | 1111 | 0001 = $4F1_{16}$

□ Conversion from binary to decimal

- $101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 5_{10}$
- $63.4_8 = 6 \times 8^1 + 3 \times 8^0 + 4 \times 8^{-1} = 51.5_{10}$
- $A1_{16} = 10 \times 16^1 + 1 \times 16^0 = 161_{10}$

Decimal → binary/octal/hex conversion

Binary

	<u>Quotient</u>	<u>Remainder</u>
$56 \div 2 =$	28	0
$28 \div 2 =$	14	0
$14 \div 2 =$	7	0
$7 \div 2 =$	3	1
$3 \div 2 =$	1	1
$1 \div 2 =$	0	1

Octal

	<u>Quotient</u>	<u>Remainder</u>
$56 \div 8 =$	7	0
$7 \div 8 =$	0	7

$56_{10} = 111000_2$

$56_{10} = 70_8$

□ Why does this work?

□ $N = 56_{10} = 111000_2$

□ $Q = N/2 = 56/2 = 111000/2 = 11100$ remainder 0

□ Each successive divide liberates an LSB (least significant bit)

Sign-and-magnitude

- The most-significant bit (MSB) is the sign digit
 - 0 $\equiv$ positive
 - 1 $\equiv$ negative
- The remaining bits are the number's magnitude
- Problem 1: Two representations for zero
 - 0 = 0000 and also $-0 = 1000$
- Problem 2: Arithmetic is cumbersome

Add

Subtract

Compare and subtract

4	0100	4	0100	0100	- 4	1100	1100
+ 3	+ 0011	- 3	+ 1011	- 0011	+ 3	+ 0011	- 0011
= 7	= 0111	= 1	$\neq$ 1111	= 0001	- 1	$\neq$ 1111	= 1001

Ones-complement

- Negative number: Bitwise complement positive number

- $0011 \equiv 3_{10}$
- $1100 \equiv -3_{10}$

- Solves the arithmetic problem

Add	Invert, add, add carry	Invert and add
$\begin{array}{r} 4 \quad 0100 \\ + 3 \quad + 0011 \\ \hline = 7 \quad = 0111 \end{array}$	$\begin{array}{r} 4 \quad 0100 \\ - 3 \quad + 1100 \\ \hline = 1 \quad 1 \ 0000 \\ \text{add carry:} \quad +1 \\ \hline = 0001 \end{array}$	$\begin{array}{r} - 4 \quad 1011 \\ + 3 \quad + 0011 \\ \hline - 1 \quad 1110 \end{array}$

- Remaining problem: Two representations for zero

- $0 = 0000$ and also $-0 = 1111$

Twos-complement

- Negative number: Bitwise complement **plus one**

- $0011 \equiv 3_{10}$
- $1101 \equiv -3_{10}$

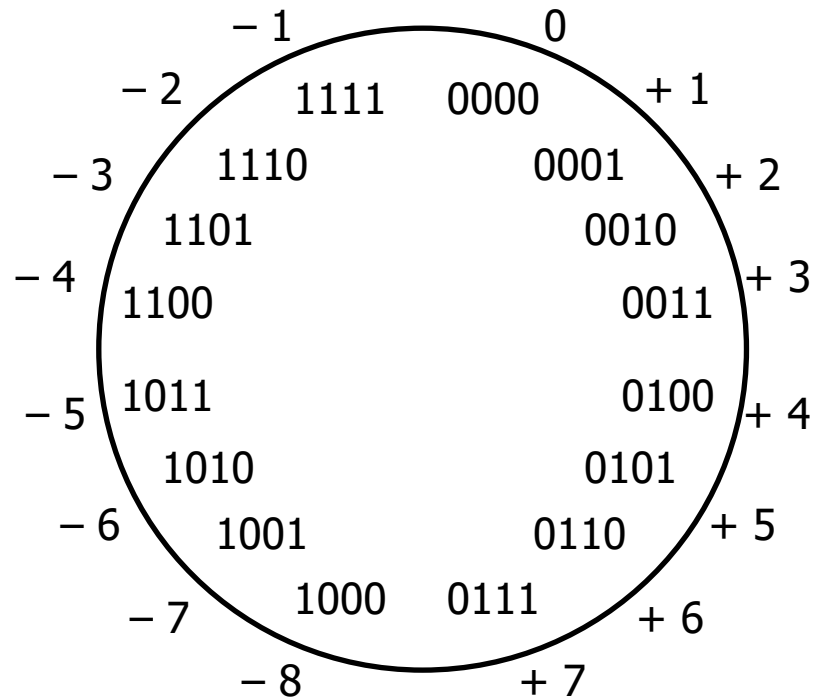
- Number wheel



- Only one zero!

- MSB is the sign digit

- $0 \equiv$ positive
- $1 \equiv$ negative



Twos-complement (con't)

- Complementing a complement □ the original number
- Arithmetic is easy
 - Subtraction = negation and addition
 - Easy to implement in hardware

Add		Invert and add		Invert and add	
4	0100	4	0100	- 4	1100
+ 3	+ 0011	- 3	+ 1101	+ 3	+ 0011
= 7	= 0111	= 1	1 0001	- 1	1111
		drop carry	= 0001		

Gray and BCD codes

<u>Decimal Symbols</u>	<u>Gray Code</u>
0	0000
1	0001
2	0011
3	0010
4	0110
5	0111
6	0101
7	0100
8	1100
9	1101

<u>Decimal Symbols</u>	<u>BCD Code</u>
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Review of Boolean algebra

□ Not is a horizontal bar above the number

□ $\neg 0 = 1$

□ $\neg 1 = 0$

□ Or is a plus

□ $0+0 = 0$

□ $0+1 = 1$

□ $1+0 = 1$

□ $1+1 = 1$

□ And is multiplication

□ $0*0 = 0$

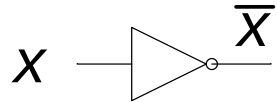
□ $0*1 = 0$

□ $1*0 = 0$

□ $1*1 = 1$

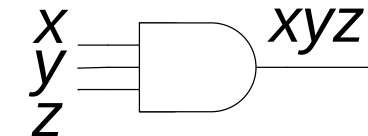
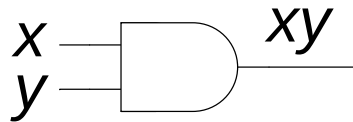
Basic logic gates

□ Not

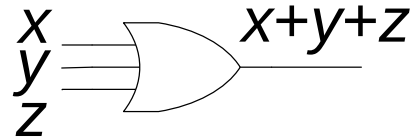
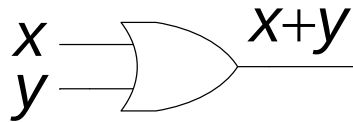


□ And

□ Or



□ Nand



□ Nor

□ Xor

